

Tutorial No. 5

Name: Varadraj Patil

Roll no: 65

Pm : 12412485

Class : CSSE-B

Problem Statement: Design appropriate behavioral UML diagrams for a Personal Expense Tracker (PET) using the provided requirements.

Solution:

Problem Statement

Design appropriate behavioral UML diagrams for a Personal Expense Tracker system (PET) using the given system requirements.

The diagrams should model:

User interactions

System workflows,

Dynamic behaviour of the application

Assignment Objectives

After completing this assignment, students should be able to:

Understand requirements and convert them into UML models.

Identify actors and system interactions

Design key behavioral UML diagrams

Model workflows, message flows, and state transitions

System Overview: PET (Personal Expense Tracker)

The personal expense tracker is a console based system used to manage daily financial

records.

The system follows operations such as:

Adding new expenses.

Viewing stored expenses.

Calculating total spending.

Setting a monthly budget.

Checking remaining balance.

Saving / loading expense data.

The system mainly supports a single user.

Assumed system Requirements.

Functional Requirements

User can add new expenses

User can enter category, amount, date, description.

User can view expenses.

System calculates the total expenses.

User can set or update monthly budget.

System shows remaining balance.

System ~~also~~ alerts when budget is exceeded.

System saves data to files.

System loads data when program starts.

Non-Functional Requirements.

System should be simple and user friendly.

System should respond quickly.

System should store data reliably in files.

System should be portable across systems with a C++ compiler.

Actors Identification

User :- Person managing personal expenses using the system.

Behavioral UML Diagrams.

Behavioral diagrams show dynamic system behavior and interaction. The following diagrams are designed for personal expense tracker:

Use Case Diagram

Purpose : Represents interaction between the user and system features.

Actor and use cases:

User can:

Add expense

View expenses

Set budget

View remaining balance

Check total spending

Save data

Load data

Description: This diagram gives a high-level overview of system functionality from the user's perspective and defines system boundaries.

Activity Diagram (Add expense process)

Purpose : Models the workflow of recording an expense.

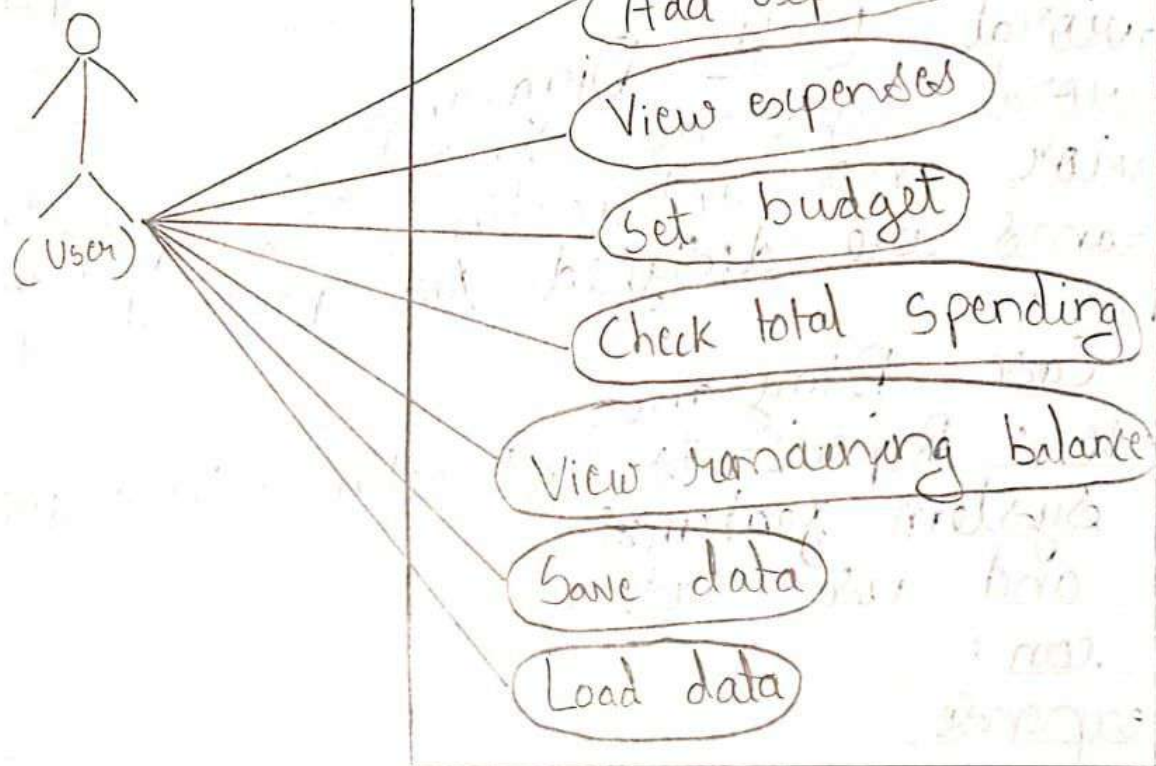
Activities:

User opens system

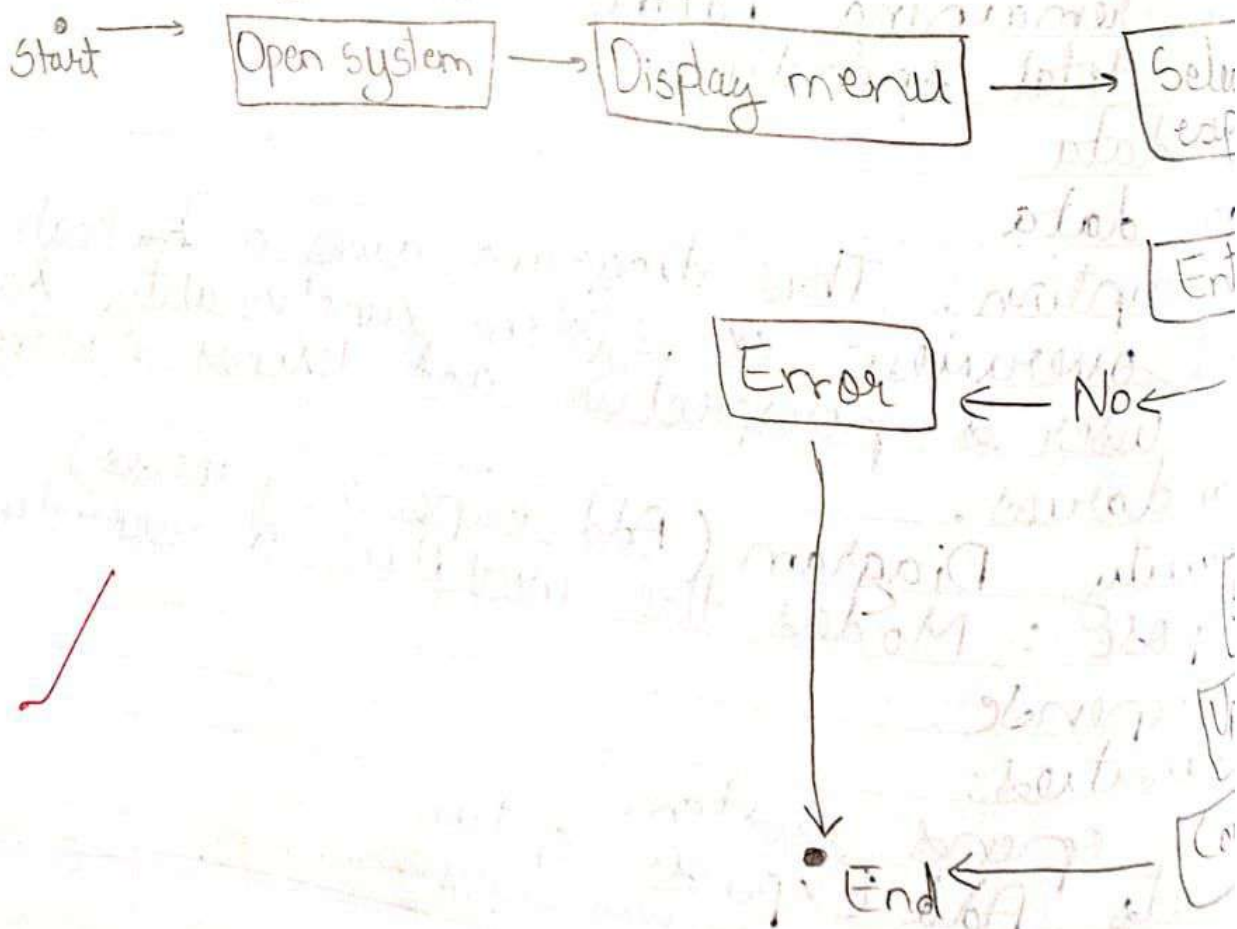
Selects Add Expense option.

38

6.1 Use Case Diagram



6.2 Activity Diagram





Enter category, amount, date, description

System validates input

Expense is stored

Total expense is updated

Confirmation is shown

Description:

This diagram shows control flow and decisions (eg., invalid amount input)

Sequence Diagram (Budget Check process)

Purpose: shows interaction between objects over time

Objects Involved:

User

Expense Tracker System

Expense list / ~~system~~ storage

Budget Manager

Interaction Flow:

User adds expense

System stores expense

System calculates new total

Budget manager checks against monthly budget

If exceeded → warning message

System displays updated balance

Description:

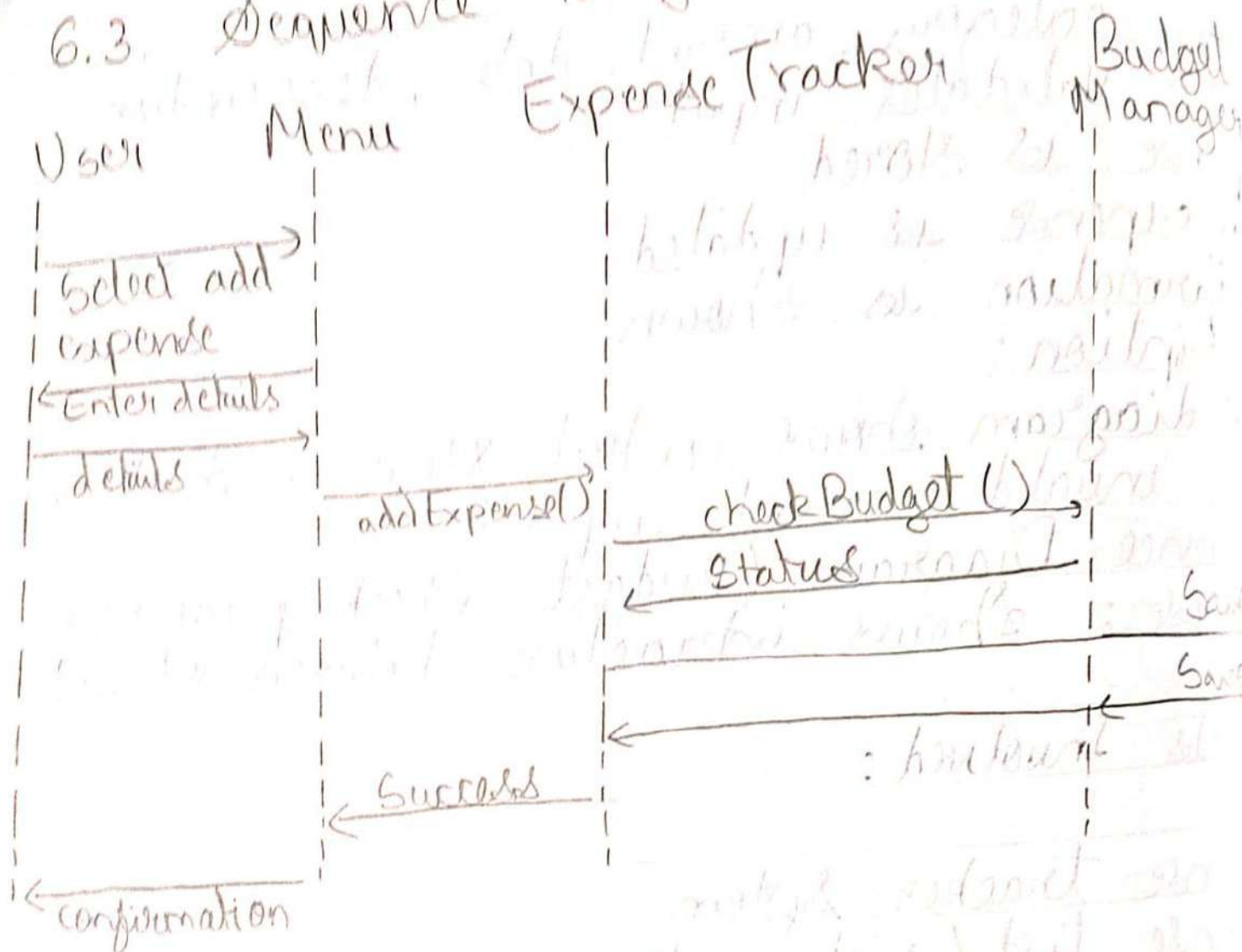
This diagram focuses on message flow and order of operations when an expense affects the budget.

System State Machine Diagram (Expense record state)

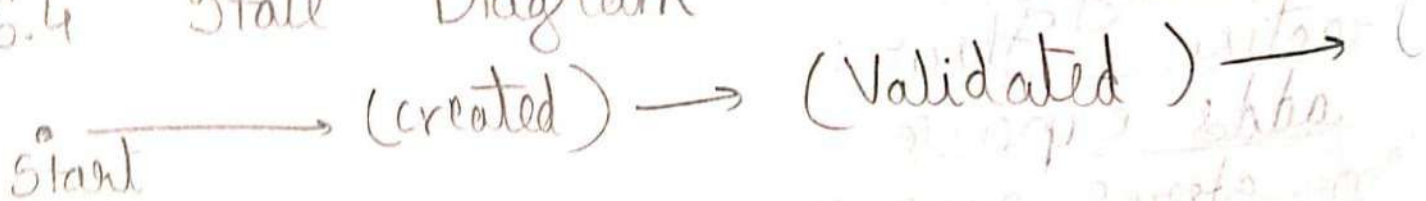
Purpose: shows lifecycle of an expense entry

50

6.3 Sequence Diagram



6.4 State Diagram



States:

Created

Stored

Included in total

Saved to file

Loaded from file

Transitions:

Created → stored (after validation)

stored → Included in total

Included → Saved to file

Saved → Loaded from file.

Description:

The diagram shows how an expense record changes state during its lifetime.

Assumptions:

- i.) User must enter valid numeric values
- ii.) Data is stored ~~to~~ locally in text files.
- iii.) Budget is set manually by the user
- iv.) Program runs in console environment.

Tools Recommended.

- i.) StarUML
- ii.) Draw.io
- iii.) Lucidchart
- iv.) Visual Paradigm

Expected Outcomes.

- Clear understanding of behavioral UML diagrams.
- Ability to model real-world systems.

Improved analysis and design skills.

Conclusion :

Behavioral UML diagrams help visualize system dynamics. This assignment demonstrates how use case, activity, sequence, and state machine diagrams together describe the behavior of a personal expense tracker system.

~~Scanned~~